User-kNN Recommendation Model

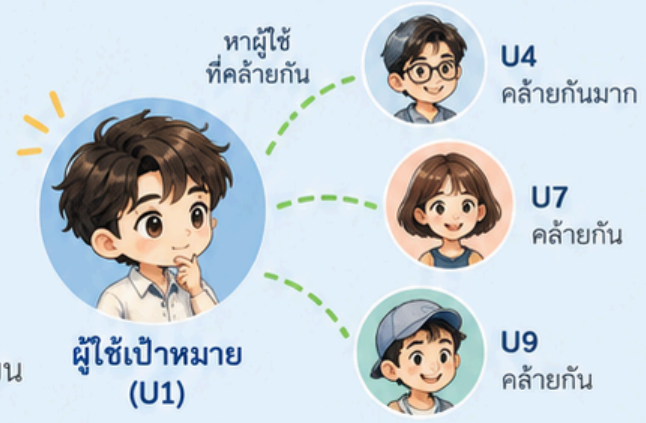
แนวคิด + ตัวอย่างการคำนวณแบบง่าย

1 User-kNN คืออะไร?

เป็นวิธีแนะนำรายการโดย “หาผู้ใช้ที่มีความคล้ายกัน” กับผู้ใช้เป้าหมาย แล้วดูว่าคนที่คล้ายกันนั้นให้คะแนนรายการนี้อย่างไร

“คนที่คล้ายคุณให้คะแนนเรื่องนี้อย่างไร”

จัดเป็นเทคนิคแบบ Memory-based Collaborative Filtering และการทำนายคะแนนจะอาศัย “ค่าความเบี่ยงเบนจากค่าเฉลี่ยของเพื่อนบ้าน” มาช่วยคำนวณ



2 แนวคิดหลัก

1. **คำนวณความคล้ายกันระหว่างผู้ใช้**
   มักใช้ Pearson Correlation ในการวัดความคล้ายกัน

2. **เลือกเพื่อนบ้าน Top-k**
   เลือกผู้ใช้ที่คล้ายกันที่สุด (ค่า similarity เป็นบวก) และที่มีการให้คะแนนรายการเป้าหมายแล้ว

3. **รวมค่าความเบี่ยงเบนและบวกค่าเฉลี่ยของผู้ใช้เป้าหมาย**
   นำค่าความเบี่ยงเบนจากค่าเฉลี่ยของเพื่อนบ้านมาถ่วงน้ำหนักและบวกกลับด้วยค่าเฉลี่ยของผู้ใช้เป้าหมาย

3 สูตรสำคัญ

$$\hat{r}_{ui} = \bar{r}_u + \frac{\sum_{v \in N_k(u;i)} sim(u, v)\,(r_{vi} - \bar{r}_v)}{\sum_{v \in N_k(u;i)} |\text{sim}(u, v)|}$$

ความหมายของสัญลักษณ์

$\bar{r}_u$	ค่าเฉลี่ยคะแนนของผู้ใช้เป้าหมาย u
$\bar{r}_v$	ค่าเฉลี่ยคะแนนของผู้ใช้เพื่อนบ้าน v
$\text{sim}(u, v)$	ค่าความคล้ายกันระหว่างผู้ใช้ u และ v (มักใช้ Pearson Correlation)
$N_k(u; i)$	กลุ่มเพื่อนบ้านจำนวน k คนของผู้ใช้ u ที่มีการให้คะแนนรายการ i แล้ว

4 ตัวอย่างการคำนวณอย่างง่าย

ต้องการทำนายคะแนนของผู้ใช้ U1 สำหรับรายการ I8 โดยใช้เพื่อนบ้าน 2 คน คือ U4 และ U7

ผู้ใช้ v	sim(U1, v)	ค่าเฉลี่ย $\bar{r}_v$	คะแนน $r_{v,I8}$	ความเบี่ยงเบน $(r_{v,I8} - \bar{r}_v)$
U4	1.0	3.75	5	5 − 3.75 = 1.25
U7	0.5	3.5	4	4 − 3.5 = 0.5

ผู้ใช้เป้าหมาย U1 มีค่าเฉลี่ยคะแนน $\bar{r}_{U1} = 3.2$

ขั้นตอนการคำนวณ

1. หาผลรวมถ่วงน้ำหนักของความเบี่ยงเบน
   $1.0 \times 1.25 + 0.5 \times 0.5 = 1.25 + 0.25 = 1.50$
2. หาผลรวมค่าสัมบูรณ์ของความคล้ายกัน
   $1.0 + 0.5 = 1.5$
3. คำนวณคะแนนที่คาดหมาย
   $\hat{r}_{U1,I8} = 3.2 + \frac{1.50}{1.5} = 3.2 + 1.0 = \mathbf{4.2}$

คะแนนที่คาดหมาย = 4.20

แนะนำรายการนี้ให้กับ U1 ได้อย่างมั่นใจ (คะแนนค่อนข้างสูง)

5 สรุปสั้น ๆ

User-kNN เป็นวิธีที่เข้าใจง่ายและอธิบายผลได้ เพราะอาศัยผู้ใช้ที่คล้ายกันในการแนะนำรายการ แต่อาจมีความไวต่อสถานการณ์ที่มีเพื่อนบ้านที่คล้ายกันจำนวนน้อย ทำให้ผลการทำนายอาจไม่เสถียร

Item-kNN Recommendation Model

แนวคิด + ตัวอย่างการคำนวณแบบง่าย

1 Item-kNN คืออะไร?

- เป็นโมเดลแนะนำรายการ (Item-kNN) ที่แนะนำรายการตามความคล้ายกันของรายการ (item) กับรายการเป้าหมาย
- ตั้งคำถามว่า **"คุณให้คะแนนรายการที่คล้ายกันอย่างไร"**
- จัดเป็นวิธีแบบ Memory-based Collaborative Filtering ที่อาศัยข้อมูลการให้คะแนนของผู้ใช้จริง
- มักคำนวณความคล้ายกันของรายการล่วงหน้า (precompute)

2 แนวคิดหลักของ Item-kNN

1 คำนวณความคล้ายกันระหว่างรายการ (เช่น Adjusted Cosine)

→

2 เลือกเพื่อนบ้านรายการที่คล้ายกันที่สุด k รายการ (เฉพาะรายการที่ผู้ใช้คนนั้นเคยให้คะแนน และมีความคล้ายกันเป็นบวก)

→

3 คาดคะเนคะแนนรายการจากคะแนนของผู้ใช้เองในรายการเพื่อนบ้านเหล่านั้น

3 สูตรสำคัญ

3.1 ความคล้ายกันระหว่างรายการ (Adjusted Cosine)

ใช้คะแนนที่ถูกปรับด้วยค่าเฉลี่ยของผู้ใช้แต่ละคน โดยพิจารณาเฉพาะผู้ใช้ที่ให้คะแนนทั้งสองรายการร่วมกัน

$$s_{ij} = \frac{\sum_{u \in U_{ij}} (r_{ui} - \bar{r}_u)(r_{uj} - \bar{r}_u)}{\sqrt{\sum_{u \in U_{ij}} (r_{ui} - \bar{r}_u)^2} \sqrt{\sum_{u \in U_{ij}} (r_{uj} - \bar{r}_u)^2}}$$

โดย r_{ui} คือคะแนนของผู้ใช้ u ต่อรายการ i
$\bar{r}_u$ คือค่าเฉลี่ยคะแนนของผู้ใช้ u
U_{ij} คือกลุ่มผู้ใช้ที่ให้คะแนนทั้งรายการ i และ j

3.2 การคาดคะเนคะแนน (Prediction)

คาดคะเนคะแนนของผู้ใช้ u สำหรับรายการ i จากคะแนนของรายการเพื่อนบ้านที่คล้ายกัน

$$\hat{r}_{ui} = \frac{\sum_{j \in N_k(i;u)} s_{ij}\, r_{uj}}{\sum_{j \in N_k(i;u)} |s_{ij}|}$$

โดย $N_k(i;u)$ คือชุดรายการเพื่อนบ้านที่คล้ายกับรายการ i (เลือกมา k รายการ เฉพาะรายการที่ผู้ใช้ u เคยให้คะแนน และมีความคล้ายกันเป็นบวก)
s_{ij} คือค่าความคล้ายกันระหว่างรายการ i และ j
r_{uj} คือคะแนนที่ผู้ใช้ u ให้กับรายการ j

4 ตัวอย่างการคำนวณอย่างง่าย

ข้อมูลการให้คะแนนของผู้ใช้ U1

ผู้ใช้ U1

รายการ	I1	I2	I3
คะแนน	5	4	2

ค่าความคล้ายกันกับรายการเป้าหมาย I4

รายการเป้าหมาย I4

รายการ	ความคล้ายกันกับ I4 (s(I4, Ij))
I1	0.8
I2	0.5
I3	-0.2

คำนวณคะแนนที่คาดการณ์สำหรับ I4

ใช้เฉพาะรายการที่มีความคล้ายกันเป็นบวก คือ I1 และ I2

$$\hat{r}_{U1,I4} = \frac{0.8 \times 5 + 0.5 \times 4}{0.8 + 0.5} = \frac{4.0 + 2.0}{1.3} = \frac{6.0}{1.3} \approx \mathbf{4.62}$$

ผลลัพธ์และการแนะนำ รายการ I4 ได้คะแนนที่คาดการณ์สูง (≈ 4.62) จึงควรแนะนำให้ผู้ใช้ U1

5 สรุปสั้น ๆ

- Item-kNN เป็นโมเดลที่อธิบายได้ง่าย (เข้าใจง่ายและตีความผลได้ชัดเจน)
- มีความเสถียร เพราะสามารถคำนวณความคล้ายกันระหว่างรายการล่วงหน้าได้
- ทำงานได้ดีเมื่อความสัมพันธ์ระหว่างรายการมีความหมาย (เช่น สินค้าที่คล้ายกัน มักถูกให้คะแนนใกล้เคียงกัน)

Bias Recommendation Model

แนวคิด + ตัวอย่างการคำนวณแบบง่าย

คำความเอนเอียง (Baseline Predictor)

แบบจำลองอย่างง่ายที่ใช้ค่าเฉลี่ยรวม μ บวกกับค่าความเอนเอียงของผู้ใช้ b_u และค่าความเอนเอียงของรายการ b_i เพื่อทำนายคะแนนความชอบ

1 ค่าความเอนเอียงคืออะไร?

แบบจำลองค่าความเอนเอียง (Baseline Predictor) เป็นแบบจำลองอย่างง่ายที่ทำนายคะแนนความชอบจากค่าเฉลี่ยรวมของข้อมูล μ บวกกับค่าความเอนเอียงของผู้ใช้ b_u และค่าความเอนเอียงของรายการ b_i

- ค่าความเอนเอียงของผู้ใช้ (b_u) สะท้อนว่าผู้ใช้มีแนวโน้มให้คะแนน "สูงกว่าปกติ" หรือ "ต่ำกว่าปกติ"
- ค่าความเอนเอียงของรายการ (b_i) สะท้อนว่ารายการนั้นได้รับความนิยมมากหรือน้อย หรือเป็นรายการที่ให้คะแนนได้ยาก
- แม้จะเรียบง่าย แต่เป็นจุดเริ่มต้นที่ดีและมักใช้เป็น baseline ที่แข็งแรงในการทำระบบแนะนำ

2 แนวคิดหลัก

1. เริ่มจากค่าเฉลี่ยรวม μ
2. ประมาณค่าความเอนเอียงของรายการ b_i (พร้อม Shrinkage)
3. ประมาณค่าความเอนเอียงของผู้ใช้ b_u
4. ทำนายคะแนนด้วยสมการ $\mu + b_u + b_i$

ความหมายของสัญลักษณ์

- μ : ค่าเฉลี่ยรวมของคะแนนทั้งหมด
- b_u : ค่าความเอนเอียงของผู้ใช้
- b_i : ค่าความเอนเอียงของรายการ
- λ : พารามิเตอร์สำหรับ Shrinkage ช่วยลดความเอนเอียงเมื่อมีข้อมูลน้อย

3 สูตรสำคัญ

ค่าความเอนเอียงของรายการ (Item Bias)

$$b_i = \frac{\sum_{u \in U_i} (r_{ui} - \mu)}{\lambda_i + |U_i|}$$

- U_i : เซตของผู้ใช้ที่ให้คะแนนรายการ i
- λ_i : พารามิเตอร์สำหรับการ Shrinkage (มักใช้ค่าเดียวกัน λ)

ค่าความเอนเอียงของผู้ใช้ (User Bias)

$$b_u = \frac{\sum_{i \in I_u} (r_{ui} - \mu - b_i)}{\lambda_u + |I_u|}$$

- I_u : เซตของรายการที่ผู้ใช้ u ให้คะแนน
- λ_u : พารามิเตอร์สำหรับการ Shrinkage (มักใช้ค่าเดียวกัน λ)

สูตรทำนายคะแนน

$$\hat{r}_{ui} = \mu + b_u + b_i$$

λ ช่วยทำให้ค่าความเอนเอียงหดเข้าหา 0 เมื่อมีจำนวนข้อมูลน้อย ป้องกันการ overfit

4 ตัวอย่างการคำนวณอย่างง่าย

1 คำนวณค่าความเอนเอียงของรายการ I_3

กำหนดให้ $\mu = 3.5$, $\lambda = 2$
รายการ I_3 มีผู้ให้คะแนน 2 คน คือ $U_2 = 5$ และ $U_3 = 4$

$$b_{I_3} = \frac{(5 - 3.5) + (4 - 3.5)}{2 + 2} = \frac{1.5 + 0.5}{4} = 0.5$$

2 คำนวณค่าความเอนเอียงของผู้ใช้ U_1

สมมติให้ผู้ใช้ U_1 ให้คะแนน $I_1 = 5$ และ $I_2 = 4$ และทราบค่าความเอนเอียงของรายการ $b_{I_1} = 0.2$ และ $b_{I_2} = 0.1$

$$b_{U_1} = \frac{(5 - 3.5 - 0.2) + (4 - 3.5 - 0.1)}{2 + 2} = \frac{1.3 + 0.4}{4} = 0.425$$

3 ทำนายคะแนนของ U_1 สำหรับ I_3

ใช้สูตร $\hat{r}_{ui} = \mu + b_u + b_i$
แทนค่า $\mu = 3.5$, $b_{U_1} = 0.425$, $b_{I_3} = 0.5$

$$\hat{r}_{U_1,I_3} = 3.5 + 0.425 + 0.5 = 4.425$$

ดังนั้น $\hat{r}_{U_1,I_3} = 4.425 \approx 4.43$

5 สรุปสั้น ๆ

- แบบจำลองค่าความเอนเอียง (Bias Model) เป็นแบบจำลองที่เรียบง่าย
- ตีความได้ง่าย เพราะค่าความเอนเอียงของผู้ใช้และรายการมีความหมายชัดเจน
- คำนวณได้รวดเร็ว และมักใช้เป็น baseline หรือเป็นส่วนประกอบในแบบจำลอง MF ที่ซับซ้อนขึ้น เช่น Matrix Factorization (SGD, ALS)

MF Recommendation Model

แนวคิด + ตัวอย่างการคำนวณแบบง่าย

1 MF คืออะไร?

MF (Matrix Factorization) คือ วิธีการแยกเมทริกซ์สำหรับระบบแนะนำที่ใช้แนวคิด Collaborative Filtering โดยทำการแยกเมทริกซ์คะแนน R ออกเป็นเมทริกซ์ของปัจจัยแฝงของผู้ใช้ P และปัจจัยแฝงของสินค้า Q ผู้ใช้และสินค้าแต่ละตัวแทนด้วยเวกเตอร์สั้น ๆ (latent vector) และทำนายคะแนนที่ยังไม่รู้ค่าด้วยการคูณจุด (dot product) ของเวกเตอร์ทั้งสอง

$$R_{(m \times n)} \approx P_{(m \times k)} \times Q^T_{(k \times n)}$$

R	P	Q^T
(m × n) เมทริกซ์คะแนน ผู้ใช้ × สินค้า	(m × k) ปัจจัยแฝงของผู้ใช้	(k × n) ปัจจัยแฝงของสินค้า
เมทริกซ์เดิม (ข้อมูลแบบกระจัดกระจาย)	เวกเตอร์ผู้ใช้ (ความชอบแฝง)	เวกเตอร์สินค้า (คุณลักษณะแฝง)

2 แนวคิดหลัก

เริ่มจากเมทริกซ์คะแนนที่มีค่าบางส่วน เรียนรู้ปัจจัยแฝงของผู้ใช้และสินค้า ทำนายคะแนน และจัดอันดับสินค้า

1 เมทริกซ์คะแนนแบบกระจัดกระจาย

R (ผู้ใช้ × สินค้า)

	I1	I2	I3	I4
U1	5	?	3	?
U2	?	4	?	1
U3	1	?	?	4
U4	?	2	5	?

→

2 เรียนรู้ปัจจัยแฝง

P (m × k) และ Q (n × k)

p_{u1} q_{i1}
p_{u2} q_{i2}
p_{u3} q_{i3}
p_{u4} q_{i4}
⋮ ⋮

เวกเตอร์ผู้ใช้แต่ละคน | เวกเตอร์สินค้าแต่ละชิ้น

→

3 ทำนายคะแนนและจัดอันดับสินค้า

$$\hat{R} \approx PQ^T$$

↓

แนะนำสินค้าที่ยังไม่เคยให้คะแนน (จัดอันดับตามคะแนน)

3 สูตรสำคัญ

1 การแยกเมทริกซ์คะแนน

$$R \approx PQ^T$$

R : เมทริกซ์คะแนน (m × n)
P : เมทริกซ์ปัจจัยแฝงของผู้ใช้ (m × k)
Q : เมทริกซ์ปัจจัยแฝงของสินค้า (n × k)

2 การทำนายคะแนนของผู้ใช้ u สำหรับสินค้า i

$$\hat{r}_{ui} = p_u^T q_i$$

โดย p_u และ q_i คือเวกเตอร์ปัจจัยแฝงขนาด k

3 ฟังก์ชันเป้าหมายในการเรียนรู้ (มี Regularization)

$$\min \sum_{(u,i)\in K} (r_{ui} - p_u^T q_i)^2 + \lambda \left(\sum_u \|p_u\|^2 + \sum_i \|q_i\|^2 \right)$$

K คือ เซตของคู่ (u, i) ที่มีคะแนนจริง (observed ratings) เท่านั้น

4 ตัวอย่างการคำนวณอย่างง่าย

กำหนดเวกเตอร์ปัจจัยแฝง (latent vector) ดังนี้

ผู้ใช้ U1

p_{U1} = [0.8, 0.2]

สินค้า (ไอเทม)

q_{I1} = [0.9, 0.1]
q_{I2} = [0.4, 0.6]
q_{I3} = [0.2, 0.7]
q_{I4} = [0.7, 0.3]

คำนวณคะแนนที่ทำนายได้สำหรับผู้ใช้ U1 (ใช้ dot product)

I1 $\hat{r}_{U1,I1} = 0.8 \times 0.9 + 0.2 \times 0.1 = 0.72 + 0.02 = 0.74$
I2 $\hat{r}_{U1,I2} = 0.8 \times 0.4 + 0.2 \times 0.6 = 0.32 + 0.12 = 0.44$
I3 $\hat{r}_{U1,I3} = 0.8 \times 0.2 + 0.2 \times 0.7 = 0.16 + 0.14 = 0.30$
I4 $\hat{r}_{U1,I4} = 0.8 \times 0.7 + 0.2 \times 0.3 = 0.56 + 0.06 = 0.62$

ผลการจัดอันดับสินค้า
(คะแนนที่ทำนายได้)

อันดับ	สินค้า	คะแนน
1	I1	0.74
2	I4	0.62
3	I2	0.44
4	I3	0.30

หากพิจารณาเฉพาะสินค้าที่ผู้ใช้ U1 ยังไม่เคยให้คะแนน **ระบบจะแนะนำ I4 เป็นอันดับแรก** (คะแนน 0.62)

5 สรุปสั้น ๆ

- MF คือการแยกเมทริกซ์คะแนนเพื่อเรียนรู้ปัจจัยแฝงของผู้ใช้และสินค้า
- ทำนายคะแนนด้วยการคูณจุดของเวกเตอร์ปัจจัยแฝง
- เหมาะกับข้อมูลที่มีคะแนนแบบกระจัดกระจาย (sparse data)
- เป็นวิธีการแบบ Model-based Collaborative Filtering ที่ได้รับความนิยมและเป็นพื้นฐานสำคัญของระบบแนะนำสมัยใหม่

SGD Recommendation Model

แนวคิด + ตัวอย่างการคำนวณแบบง่าย

1 SGD ใน MF คืออะไร?

- SGD = Stochastic Gradient Descent
- ใช้ฝึก Matrix Factorization (MF) เพื่อเรียนรู้ latent factors ของผู้ใช้และรายการ
- พยากรณ์คะแนนด้วย $\hat{r}_{ui} = p_u^T q_i$
- อัปเดตพารามิเตอร์ที่ละตัวอย่างข้อมูล (user, item, rating)
- เป้าหมายคือทำให้คะแนนที่ทำนายใกล้เคียงคะแนนจริง และลด overfitting ด้วย regularization

เมทริกซ์คะแนนผู้ใช้-รายการ R (Rating Matrix R)

	I1	I2	I3	I4
U1	5	3	0	1
U2	4	0	1	0
U3	0	2	4	0

$R \in \mathbb{R}^{m \times n}$

เมทริกซ์คะแนน (มีค่าบางส่วนเท่านั้น)

$\approx$

เวกเตอร์แฝงของผู้ใช้ P (User factors P)

	f_1	f_2	⋯	f_k
U1	0.6	0.2	⋯	
U2	0.1	0.8	⋯	
U3	0.7	0.4	⋯	

$P \in \mathbb{R}^{m \times k}$

latent factors ของผู้ใช้

$\times$

เวกเตอร์แฝงของรายการ Q (Item factors Q)

	f_1	f_2	⋯	f_k
I1	0.5	0.4	⋯	
I2	0.1	0.9	⋯	
I3	0.8	0.1	⋯	
I4	0.4	0.3	⋯	

$Q \in \mathbb{R}^{n \times k}$

latent factors ของรายการ

$\rightarrow$

คะแนนที่ทำนายได้ $\hat{R}$ (Predicted Scores)

	I1	I2	I3	I4
U1	2.8	2.1	1.5	1.2
U2	1.1	3.0	2.3	1.8
U3	2.5	2.0	3.6	1.9

$\hat{R} = PQ^T$

$\hat{R} \in \mathbb{R}^{m \times n}$

คะแนนที่ทำนายได้สำหรับทุกคู่ (ผู้ใช้, รายการ)

2 แนวคิดหลัก

1. เก็บข้อมูล rating

	I1	I2	I3	I4
U1	5	3	0	1
U2	4	0	1	0
U3	0	2	4	0

R

2. กำหนด latent factors k และสุ่มค่าเริ่มต้น

$p_u \in \mathbb{R}^k$, $q_i \in \mathbb{R}^k$

กำหนด k และสุ่มค่าเริ่มต้น

3. คำนวณคะแนนที่ทำนาย

$\hat{r}_{ui} = p_u^T q_i$

ใช้เวกเตอร์แฝงของผู้ใช้และรายการคำนวณคะแนน

4. หาความคลาดเคลื่อน

$e_{ui} = r_{ui} - \hat{r}_{ui}$

ความแตกต่างระหว่างคะแนนจริงและคะแนนที่ทำนาย

5. อัปเดตพารามิเตอร์ด้วย SGD

$p_u \leftarrow p_u + \eta(e_{ui} q_i - \lambda p_u)$

$q_i \leftarrow q_i + \eta(e_{ui} p_u - \lambda q_i)$

ปรับค่า p_u และ q_i จากความคลาดเคลื่อน

6. ทำซ้ำหลายรอบ แล้วใช้คะแนนแนะนำ

ฝึกหลายรอบด้วยข้อมูลจำนวนมาก แล้วใช้ $\hat{r}_{ui}$ เพื่อจัดลำดับและแนะนำรายการ

ความหมาย

- R = เมทริกซ์คะแนนผู้ใช้-รายการ
- p_u = เวกเตอร์แฝงของผู้ใช้ u
- q_i = เวกเตอร์แฝงของรายการ i
- k = จำนวนมิติแฝง (latent dimension)
- η = learning rate
- λ = regularization

3 สูตรสำคัญ

1. **Prediction**

$$\hat{r}_{ui} = p_u^T q_i$$

คะแนนที่ทำนายของผู้ใช้ u สำหรับรายการ i

2. **Error**

$$e_{ui} = r_{ui} - \hat{r}_{ui}$$

ความคลาดเคลื่อนระหว่างคะแนนจริงและคะแนนที่ทำนาย

3. **Loss (with regularization)**

$$L = \frac{1}{2}(r_{ui} - \hat{r}_{ui})^2 + \frac{\lambda}{2}(\|p_u\|^2 + \|q_i\|^2)$$

ฟังก์ชันความสูญเสียที่ต้องการลด

4. **Update user factor**

$$p_u \leftarrow p_u + \eta(e_{ui} q_i - \lambda p_u)$$

อัปเดตเวกเตอร์แฝงของผู้ใช้ด้วย SGD

5. **Update item factor**

$$q_i \leftarrow q_i + \eta(e_{ui} p_u - \lambda q_i)$$

อัปเดตเวกเตอร์แฝงของรายการด้วย SGD

SGD จะค่อย ๆ ปรับ latent factors ของผู้ใช้และรายการจากข้อผิดพลาดของแต่ละตัวอย่าง เพื่อให้คะแนนที่ทำนายดีขึ้นเรื่อย ๆ

4 ตัวอย่างการคำนวณอย่างง่าย

ข้อมูลตัวอย่าง

ผู้ใช้ U1 ให้คะแนนรายการ $I_1 = 5$

กำหนด $k = 2$, $\eta = 0.1$, $\lambda = 0$

คำเริ่มต้น:

$p_{U1} = [0.6, 0.2]$

$q_{I1} = [0.5, 0.4]$

U1, I1

★★★★★

1 คำนวณคะแนนที่ทำนาย

$$\hat{r}_{U1,I1} = p_{U1}^T q_{I1}$$
$$= (0.6 \times 0.5) + (0.2 \times 0.4)$$
$$= 0.30 + 0.08$$
$$= \mathbf{0.38}$$

2 คำนวณ error

$$e_{U1,I1} = r_{U1,I1} - \hat{r}_{U1,I1}$$
$$= 5 - 0.38$$
$$= \mathbf{4.62}$$

3 อัปเดตเวกเตอร์ผู้ใช้

$p_{U1(new)} = p_{U1} + \eta(eq)$

$= [0.6, 0.2] + 0.1 \times 4.62 \times [0.5, 0.4]$

$= [0.6, 0.2] + [0.231, 0.1848]$

$= \mathbf{[0.831, 0.3848]}$

4 อัปเดตเวกเตอร์รายการ

$q_{I1(new)} = q_{I1} + \eta(ep)$

$= [0.5, 0.4] + 0.1 \times 4.62 \times [0.6, 0.2]$

$= [0.5, 0.4] + [0.2772, 0.0924]$

$= \mathbf{[0.7772, 0.4924]}$

5 คำนวณคะแนนใหม่หลังอัปเดต

$\hat{r}_{new} = [0.831, 0.3848] \cdot [0.7772, 0.4924]$

$\approx 0.646 + 0.189$

$= \mathbf{0.835}$

ผลลัพธ์การเรียนรู้

แม้ยังไม่ถึง 5 แต่โมเดลขยับพารามิเตอร์ไปในทิศทางที่ทำให้ค่าทำนายสูงขึ้น

เมื่อฝึกหลายรอบกับข้อมูลจำนวนมาก โมเดลจะเรียนรู้ latent factors ที่ดีขึ้นและใช้จัดอันดับรายการเพื่อแนะนำได้

5 สรุปสั้น ๆ

1. SGD ฝึก MF แบบค่อย ๆ ปรับค่าที่ละตัวอย่าง
2. ใช้ error เป็นสัญญาณในการอัปเดต latent factors
3. มี learning rate และ regularization ช่วยให้เรียนรู้เสถียรขึ้น
4. หลังฝึกแล้ว สามารถคำนวณคะแนนและจัดอันดับเพื่อแนะนำ top-N ได้

แนวคิดสำคัญ: ทำนาย → วัด error → อัปเดต p และ q → ทำซ้ำจนโมเดลดีขึ้น

ALS Recommendation Model

แนวคิด + ตัวอย่างการคำนวณแบบง่าย

1 ALS ใน MF คืออะไร?

- ALS = Alternating Least Squares
- ใช้ร่วมกับ Matrix Factorization (MF) เพื่อแยกเมทริกซ์คะแนนผู้ใช้-รายการ
- เรียนรู้ latent factors ของผู้ใช้และรายการ
- สลับแก้สมการแบบ least squares: ตรึง Q แล้วหา P, จากนั้นตรึง P แล้วหา Q
- เหมาะกับข้อมูลขนาดใหญ่ และทำงานได้ดีกับเมทริกซ์ที่เบาบาง (sparse)

เมทริกซ์คะแนนผู้ใช้-รายการ R (Rating Matrix R)

	I1	I2	I3	I4
U1	5	3	0	0
U2	4	0	1	0
U3	0	2	4	0

$\approx$

เวกเตอร์แฝงของผู้ใช้ P (User factors P)

	f_1	f_2	⋯	f_k
U1	0.2	1.1	⋯	
U2	1.5	0.3	⋯	
U3	0.7	0.4	⋯	

$\times$

เวกเตอร์แฝงของรายการ Q (Item factors Q)

	f_1	f_2	⋯	f_k
I1	1.0	0.2	⋯	
I2	0.1	0.9	⋯	
I3	0.8	0.6	⋯	
I4	0.4	0.9	⋯	

$\Rightarrow$

คะแนนที่ทำนายได้ $\hat{R}$ (Predicted Scores)

	I1	I2	I3	I4
U1	3.2	2.8	4.1	1.7
U2	1.1	3.9	2.0	2.4
U3	2.5	3.1	4.0	3.6

$R \in \mathbb{R}^{m \times n}$ เมทริกซ์คะแนน (มีค่าบางส่วนเท่านั้น)

$P \in \mathbb{R}^{m \times k}$ latent factors ของผู้ใช้

$Q \in \mathbb{R}^{n \times k}$ latent factors ของรายการ

$\hat{R} = PQ^T$, $\hat{R} \in \mathbb{R}^{m \times n}$ คะแนนที่ทำนายได้สำหรับทุกคู่ (ผู้ใช้, รายการ)

2 แนวคิดหลัก

1 เก็บข้อมูล rating

	I1	I2	I3	I4
U1	5	3	0	0
U2	4	0	1	0
U3	0	2	4	0

R

2 กำหนด latent factors k

$p_u \in \mathbb{R}^k$

$q_i \in \mathbb{R}^k$

กำหนดจำนวนมิติแฝง เช่น $k = 2, 5, 10$

3 ตรึง Q แล้วอัปเดต P

$$p_u = (Q_u^T Q_u + \lambda I)^{-1} Q_u^T r_u$$

แก้สมการ least squares เพื่อหา p_u สำหรับผู้ใช้แต่ละคน

4 ตรึง P แล้วอัปเดต Q

$$q_i = (P_i^T P_i + \lambda I)^{-1} P_i^T r_i$$

แก้สมการ least squares เพื่อหา q_i สำหรับรายการแต่ละชิ้น

5 คำนวณ error และทำซ้ำ

สลับอัปเดตจนกว่าจะลู่เข้า

6 ทำนายคะแนนและจัดลันดับ

$$\hat{r}_{ui} = p_u^T q_i$$

คำนวณคะแนนและแนะนำรายการที่ดีที่สุด

ความหมาย

- R = เมทริกซ์คะแนนผู้ใช้-รายการ
- P = เวกเตอร์แฝงของผู้ใช้
- Q = เวกเตอร์แฝงของรายการ
- k = จำนวนมิติแฝง (latent factors)
- λ = ค่าควบคุมการ overfitting (regularization)

3 สูตรสำคัญ

1 Prediction

$$\hat{r}_{ui} = p_u^T q_i$$

คะแนนที่ทำนายของผู้ใช้ u สำหรับรายการ i

2 Objective

$$\min_{P,Q} \sum_{(u,i)\in\Omega} (r_{ui} - p_u^T q_i)^2 + \lambda \left(\sum_u \|p_u\|^2 + \sum_i \|q_i\|^2 \right)$$

ฟังก์ชันวัดฤประสงค์ของ ALS (มี regularization)

3 อัปเดตผู้ใช้

$$\hat{p}_u = (Q_u^T Q_u + \lambda I)^{-1} Q_u^T r_u$$

อัปเดตเวกเตอร์ผู้ใช้ u โดยตรึง Q

4 อัปเดตรายการ

$$q_i = (P_i^T P_i + \lambda I)^{-1} P_i^T r_i$$

อัปเดตเวกเตอร์รายการ i โดยตรึง P

5 แนวคิดสำคัญ

ALS ไม่ใช้ gradient descent โดยตรง แต่แก้ปัญหา least squares แบบสลับฝั่งระหว่างผู้ใช้และรายการ

ALS จะพยายามหาปัจจัยแฝงที่ทำให้คะแนนที่ทำนายใกล้เคียงคะแนนจริงมากที่สุด พร้อมลดความซับซ้อนของโมเดลด้วย regularization

4 ตัวอย่างการคำนวณอย่างง่าย

A ข้อมูลตัวอย่าง

ผู้ใช้ U1 ให้คะแนน: $I1 = 5$, $I_2 = 3$

รายการที่ยังไม่เคยให้คะแนน: $I3$, I_4

กำหนด $k = 2$ และ $\lambda = 1$

กำหนดเวกเตอร์รายการที่ทราบแล้ว

เวกเตอร์รายการ (item factors Q)

$q_{I1} = [1, 0]$ $q_{I2} = [0, 1]$

$q_{I3} = [0.8, 0.6]$ $q_{I4} = [0.4, 0.9]$

เวกเตอร์คะแนนของผู้ใช้ U1 (เฉพาะรายการที่ให้คะแนน)

$$r_{U1} = [5, 3]^T$$

1 คำนวณ $Q_u^T Q_u$

สำหรับ U1 มีรายการที่ให้คะแนน: $I1, I_2$

$$Q_u = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$Q_u^T Q_u = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

2 เพิ่ม regularization

$$Q_u^T Q_u + \lambda I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} + 1 \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$$

3 หา inverse และอัปเดตผู้ใช้

$$(Q_u^T Q_u + \lambda I)^{-1} = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix}$$

$$p_{U1} = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} 5 \\ 3 \end{bmatrix} = \begin{bmatrix} 2.5 \\ 1.5 \end{bmatrix}$$

เวกเตอร์ผู้ใช้ที่ได้ $p_{U1} = [2.5, 1.5]$

4 ทำนายคะแนนของรายการที่ยังไม่เคยให้คะแนน

$$\hat{r}_{U1,I3} = [2.5, 1.5] \cdot [0.8, 0.6] = 2.0 + 0.9 = \mathbf{2.9}$$

$$\hat{r}_{U1,I4} = [2.5, 1.5] \cdot [0.4, 0.9] = 1.0 + 1.35 = \mathbf{2.35}$$

5 ผลลัพธ์การแนะนำ

อันดับ	รายการ	คะแนนที่ทำนาย
1	I3	2.90
2	I4	2.35

ดังนั้น ระบบแนะนำ I3 ก่อน เพราะ I3 มีคะแนนทำนายสูงกว่า I4

เมื่อมีผู้ใช้และรายการมากขึ้น ALS จะสลับอัปเดตผู้ใช้ทุกรายและรายการทุกชิ้นหลายรอบ

5 สรุปสั้น ๆ

1. ALS เป็นวิธีฝึก MF แบบสลับแก้สมการ
2. เรียนรู้ latent factors ของผู้ใช้และรายการ
3. ทำนายคะแนนด้วย $\hat{r}_{ui} = p_u^T q_i$
4. หลังฝึกแล้ว สามารถจัดอันดับรายการเพื่อแนะนำ top-N ได้

แนวคิดสำคัญ: ตรึงอีกฝั่งไว้ แล้วแก้ least squares ทีละฝั่ง → ทำซ้ำจนได้ตัวแทนผู้ใช้และรายการที่ดี

SVD++ Recommendation Model

แนวคิด + ตัวอย่างการคำนวณแบบง่าย

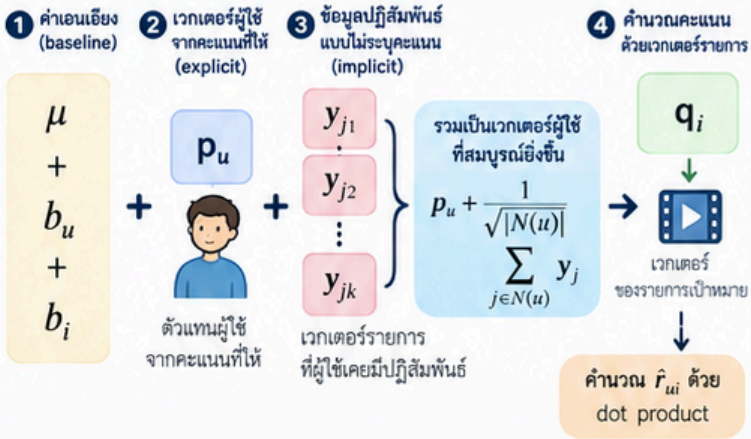
ใช้ข้อมูลการให้คะแนน ร่วมกับ ข้อมูลการมีปฏิสัมพันธ์โดยไม่ระบุคะแนน
เพื่อสร้างตัวแทนผู้ใช้ที่สมบูรณ์ยิ่งขึ้น แถให้คำแนะนำที่แม่นยำกว่าเดิม

1 SVD++ คืออะไร?

SVD++ เป็นวิธีที่ต่อยอดจาก Matrix Factorization (MF) โดยเพิ่มข้อมูลการมีปฏิสัมพันธ์แบบไม่ระบุคะแนน (Implicit Feedback) เข้ามาใช้ ไม่ใช้แค่คะแนนที่ผู้ใช้ให้เท่านั้น แต่ยังใช้ข้อมูลว่า ผู้ใช้เคยมีปฏิสัมพันธ์กับรายการใดบ้าง (เช่น ดู คลิก ซื้อ) เพื่อสร้างตัวแทนผู้ใช้ให้มีข้อมูลมากขึ้น จึงมักให้คำแนะนำที่แม่นยำกว่า MF แบบธรรมดา

2 แนวคิดหลัก

โครงสร้างการคำนวณคะแนนใน SVD++



3 สูตรสำคัญ

สูตรการทำนายคะแนนของ SVD++

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{q}_i^T \left(\mathbf{p}_u + \frac{1}{\sqrt{|N(u)|}} \sum_{j \in N(u)} \mathbf{y}_j \right)$$

$N(u)$ = เซตของรายการที่ผู้ใช้ u เคยมีปฏิสัมพันธ์ด้วย (เช่น ดู คลิก ซื้อ) โดยไม่ระบุคะแนน

ความหมายของสัญลักษณ์

$\hat{r}_{ui}$	: คะแนนที่ทำนายของผู้ใช้ u สำหรับรายการ i
μ	: ค่าเฉลี่ยคะแนนโดยรวม (global mean)
b_u	: ค่าเอนเอียงของผู้ใช้ u
b_i	: ค่าเอนเอียงของรายการ i
$\mathbf{p}_u$	: เวกเตอร์ตัวแทนผู้ใช้จากคะแนนที่ให้ (explicit)
$\mathbf{y}_j$	: เวกเตอร์ของรายการ j ที่ผู้ใช้เคยมีปฏิสัมพันธ์ (implicit)
$\mathbf{q}_i$	: เวกเตอร์ตัวแทนของรายการ i
$N(u)$	: เซตของรายการที่ผู้ใช้ u เคยมีปฏิสัมพันธ์

4 ตัวอย่างการคำนวณอย่างง่าย

กำหนดค่า

- $\mu = 3.5$
- $b_u = 0.2$
- $b_i = 0.1$
- $\mathbf{p}_u = [0.6, 0.2]$
- $N(u) = \{I_1, I_2\}$
- $\mathbf{y}_1 = [0.2, 0.0]$
- $\mathbf{y}_2 = [0.0, 0.2]$
- $\mathbf{q}_i = [0.7, 0.3]$

1 คำนวณส่วน implicit

$\mathbf{y}_1 + \mathbf{y}_2 = [0.2, 0.2]$

$|N(u)| = 2 \Rightarrow \frac{1}{\sqrt{2}} [0.2, 0.2] \approx [0.141, 0.141]$

2 เวกเตอร์ผู้ใช้ที่เสริมข้อมูลแล้ว

$$\mathbf{p}_u + \frac{1}{\sqrt{|N(u)|}} \sum \mathbf{y}_j = [0.6, 0.2] + [0.141, 0.141] = \mathbf{[0.741, 0.341]}$$

3 คำนวณ dot product กับเวกเตอร์รายการ

$$\mathbf{q}_i^T [0.741, 0.341] = 0.741 \times 0.7 + 0.341 \times 0.3 = 0.5187 + 0.1023 = \mathbf{0.621}$$

4 ทำนายคะแนนสุดท้าย

$$\hat{r}_{ui} = 3.5 + 0.2 + 0.1 + 0.621 = 4.421 \approx \mathbf{4.42}$$

เปรียบเทียบ: ถ้าไม่ใช้ข้อมูล implicit

ส่วน dot product จะเป็นเพียง

$$0.6 \times 0.7 + 0.2 \times 0.3 = \mathbf{0.48}$$

SVD++ ใช้ข้อมูลประวัติการมีปฏิสัมพันธ์เพิ่มเติม จึงได้ค่าคะแนนที่แตกต่างและมักแม่นยำกว่า

5 สรุปสั้น ๆ

- SVD++ = Matrix Factorization (MF) + ข้อมูลประวัติการมีปฏิสัมพันธ์แบบไม่ระบุคะแนน
- โดยทั่วไปให้ผลลัพธ์ที่ดีกว่า MF แบบธรรมดา เมื่อประวัติการมีปฏิสัมพันธ์ของผู้ใช้มีความสำคัญ
- การทำนายคะแนนยังคงใช้ dot product เช่นเดิม แต่ใช้เวกเตอร์ผู้ใช้ที่มีข้อมูล richer ขึ้น จากการรวมเวกเตอร์ของรายการที่ผู้ใช้เคยมีปฏิสัมพันธ์

WRMF Recommendation Model

แนวคิด + ตัวอย่างการคำนวณแบบง่าย

1 WRMF คืออะไร?

- WRMF = Weighted Regularized Matrix Factorization
- เป็นโมเดลแนะนำสำหรับ implicit feedback เช่น click, view, play, purchase
- ไม่ใช้ rating ตรง ๆ แต่แปลงพฤติกรรมเป็น preference และ confidence
- เรียนรู้ latent factors ของผู้ใช้และรายการ
- มักฝึกด้วย Alternating Least Squares (ALS)

ภาพรวมการทำงานของ WRMF

พฤติกรรมผู้ใช้ - รายการ r_{ui} → แปลงเป็น Preference p_{ui} (1 / 0) → กำหนด Confidence c_{ui} → เรียนรู้ latent factors x_u, y_i → คำนวณคะแนน score $= x_u^T y_i$

ตัวอย่างเมทริกซ์พฤติกรรมผู้ใช้ - รายการ R (เช่น จำนวนครั้งที่ดู/คลิก)

ผู้ใช้ / รายการ	I1	I2	I3	I4
U1	3	1	0	0
U2	0	2	1	0
U3	1	0	0	4

r_{ui} คือ จำนวนครั้งที่ผู้ใช้ u มีพฤติกรรมกับรายการ i (implicit feedback)

เช่น view, click, play, purchase

2 แนวคิดหลัก

1. เก็บข้อมูล implicit feedback — เช่น click, view play, purchase
2. แปลงเป็น preference — $p_{ui} = 1$ ถ้า $r_{ui} > 0$ ไม่เช่นนั้น 0
3. กำหนด confidence — $c_{ui} = 1 + \alpha r_{ui}$
4. เรียนรู้ latent factors ของผู้ใช้และรายการ — x_u y_i
5. ฝึกโมเดลด้วย ALS — สลับอัปเดต X และ Y จนกว่าจะลู่เข้า
6. คำนวณคะแนนและจัดอันดับ — $\hat{r}_{ui} = x_u^T y_i$

ความหมาย

r_{ui}	จำนวนครั้งที่ผู้ใช้ u มีพฤติกรรมกับรายการ i
p_{ui}	ค่า preference (1 ถ้ามีพฤติกรรม, 0 ถ้าไม่มี)
c_{ui}	ค่า confidence (ความมั่นใจ)
x_u	เวกเตอร์แฝงของผู้ใช้ u (d มิติ)
y_i	เวกเตอร์แฝงของรายการ i (d มิติ)

3 สูตรสำคัญ

1 Preference

$$p_{ui} = \begin{cases} 1 & \text{ถ้า } r_{ui} > 0 \\ 0 & \text{มิฉะนั้น} \end{cases}$$

2 Confidence

$$c_{ui} = 1 + \alpha r_{ui}$$

3 Prediction

$$\hat{r}_{ui} = x_u^T y_i$$

4 Objective (ฟังก์ชันที่ต้องการลดค่า)

$$\min_{X,Y} \sum_{u,i} c_{ui}(p_{ui} - x_u^T y_i)^2 + \lambda \left(\sum_u \|x_u\|^2 + \sum_i \|y_i\|^2 \right)$$

5 ALS update ของผู้ใช้

$$x_u = \left(Y^T C^u Y + \lambda I\right)^{-1} Y^T C^u p(u)$$

WRMF ให้น้ำหนักกับข้อมูลที่มีปฏิสัมพันธ์มากกว่า ทำให้ข้อมูลที่ผู้ใช้มีพฤติกรรมบ่อยมีผลต่อการเรียนรู้มากกว่า

4 ตัวอย่างการคำนวณอย่างง่าย (สำหรับผู้ใช้ U1)

1 พฤติกรรมของผู้ใช้ U1

รายการ	I1	I2	I3	I4
r_{U1}	3	1	0	0

2 กำหนด $\alpha = 2$ และแปลงเป็น preference และ confidence

$p_{U1} = [1, 1, 0, 0]$

$c_{U1} = [7, 3, 1, 1]$

(เพราะ $c_{ui} = 1 + 2\, r_{ui}$)

3 เวกเตอร์แฝงที่เรียนรู้แล้ว

$x_{U1} = [0.8, 0.2]$

$y_{I1} = [0.9, 0.1]$

$y_{I2} = [0.4, 0.6]$

$y_{I3} = [0.2, 0.7]$

$y_{I4} = [0.7, 0.3]$

4 คำนวณคะแนน $\hat{r}_{U1,i} = x_{U1}^T y_i$

$\hat{r}_{U1,I1} = [0.8, 0.2] \cdot [0.9, 0.1] = 0.72 + 0.02 = \mathbf{0.74}$

$\hat{r}_{U1,I2} = [0.8, 0.2] \cdot [0.4, 0.6] = 0.32 + 0.12 = \mathbf{0.44}$

$\hat{r}_{U1,I3} = [0.8, 0.2] \cdot [0.2, 0.7] = 0.16 + 0.14 = \mathbf{0.30}$

$\hat{r}_{U1,I4} = [0.8, 0.2] \cdot [0.7, 0.3] = 0.56 + 0.06 = \mathbf{0.62}$

5 ตัวอย่าง weighted error

สำหรับ I1 (เคยโต้ตอบ, $p = 1$)

$$c(p - \hat{r})^2 = 7(1 - 0.74)^2 = 7(0.26)^2 = 7(0.0676) = 0.4732$$

สำหรับ I3 (ไม่เคยโต้ตอบ, $p = 0$)

$$c(p - \hat{r})^2 = 1(0 - 0.30)^2 = 0.09$$

6 ตารางจัดอันดับคะแนนสำหรับผู้ใช้ U1

อันดับ	รายการ	คะแนน $\hat{r}_{U1,i}$	เคยโต้ตอบแล้ว?
1	I1	0.74	✓ เคย
2	I4	0.62	✗ ยังไม่เคย
3	I2	0.44	✓ เคย
4	I3	0.30	✗ ยังไม่เคย

7 ผลลัพธ์การแนะนำ

เมื่อพิจารณาเฉพาะรายการที่ยังไม่เคยโต้ตอบ I4 มีคะแนนสูงกว่า I3 ดังนั้นระบบแนะนำ I4 ก่อน

แนะนำ: I4 (คะแนน 0.62)

5 สรุปสั้น ๆ

- เหมาะกับ implicit feedback
- ใช้ confidence เพื่อสะท้อนความมั่นใจ ยิ่งมีพฤติกรรมมากยิ่งมั่นใจมาก
- เรียนรู้ latent factors ของผู้ใช้และรายการ
- หลังฝึกแล้วใช้คะแนนทำนาย เพื่อแนะนำ top-N ได้

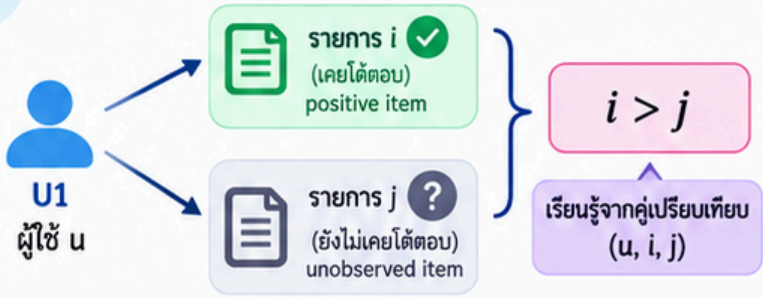
แนวคิดสำคัญ: พฤติกรรมมากกว่า → ความมั่นใจสูงกว่า → มีผลต่อการเรียนรู้มากกว่า

BPR Recommendation Model

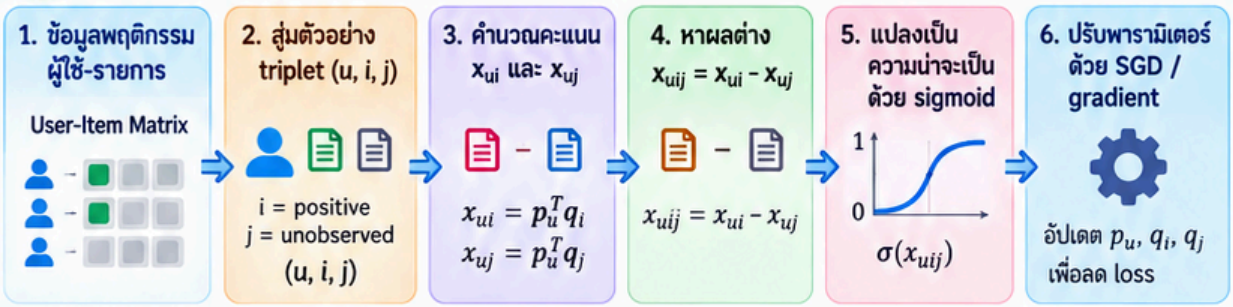
แนวคิด + ตัวอย่างการคำนวณแบบง่าย

1 BPR คืออะไร?

- BPR = Bayesian Personalized Ranking
- เป็นโมเดลแนะนำสำหรับ implicit feedback เช่น click, view, purchase
- เรียนรู้แบบ pairwise ranking ไม่ได้ทำนายคะแนนแบบตรง ๆ
- เป้าหมายคือทำให้รายการที่ผู้ใช้เคยโต้ตอบ มีอันดับสูงกว่ารายการที่ยังไม่เคยโต้ตอบ
- มักใช้ร่วมกับ latent factors / matrix factorization



2 แนวคิดหลัก



ความหมาย

- u = ผู้ใช้
- i = รายการที่ผู้ใช้เคยโต้ตอบ
- j = รายการที่ยังไม่เคยโต้ตอบ
- p_u, q_i, q_j = เวกเตอร์แฝง (latent vectors)

3 สูตรสำคัญ

1) คะแนนของผู้ใช้ต่อรายการ

$$x_{ui} = p_u^T q_i$$

2) คะแนนเปรียบเทียบ

$$x_{uij} = x_{ui} - x_{uj}$$

3) ความน่าจะเป็นที่ i ควรอยู่อันดับสูงกว่า j

$$P(i >_u j) = \sigma(x_{uij}) = \frac{1}{1 + e^{-x_{uij}}}$$

4) ฟังก์ชันเป้าหมาย

$$\text{maximize} \sum \ln \sigma(x_{uij}) - \lambda \|\Theta\|^2$$

BPR จะพยายามทำให้รายการเชิงบวกมีคะแนนสูงกว่ารายการที่ยังไม่เคยโต้ตอบ โดยเพิ่มระยะห่างของอันดับให้เหมาะสม

4 ตัวอย่างการคำนวณอย่างง่าย

ข้อมูลตัวอย่าง

- ผู้ใช้ U1 เคยโต้ตอบ: I1
- รายการที่ยังไม่เคยโต้ตอบ: I2, I3
- สุ่มคู่ฝึก: (U1, I1, I3)
- เวกเตอร์แฝงของผู้ใช้และรายการ

$$p_{U1} = [0.6, 0.2]$$
$$q_{I1} = [0.7, 0.1]$$
$$q_{I2} = [0.5, 0.3]$$
$$q_{I3} = [0.2, 0.5]$$

1 คำนวณคะแนนของ I1

$$x_{U1,I1} = p_U{}^T q_{I1}$$
$$= (0.6 \times 0.7) + (0.2 \times 0.1)$$
$$= 0.42 + 0.02 = 0.44$$

2 คำนวณคะแนนของ I3

$$x_{U1,I3} = p_U{}^T q_{I3}$$
$$= (0.6 \times 0.2) + (0.2 \times 0.5)$$
$$= 0.12 + 0.10 = 0.22$$

3 หาค่าความต่างและ sigmoid

$$x_{uij} = x_{U1,I1} - x_{U1,I3}$$
$$= 0.44 - 0.22 = 0.22$$
$$\sigma(0.22) = \frac{1}{1 + e^{-0.22}} \approx 0.555$$

ยิ่งค่าเข้าใกล้ 1 มาก แปลว่าโมเดลมั่นใจว่า I1 ควรอยู่อันดับสูงกว่า I3

4 ค่าความสูญเสียของคู่ฝึก

$$L = -\ln \sigma(x_{uij})$$
$$= -\ln(0.555)$$
$$\approx 0.589$$

ระหว่างการฝึก โมเดลจะปรับพารามิเตอร์เพื่อให้ loss ลดลง

5 จัดอันดับรายการที่ยังไม่เคยโต้ตอบ

$$x_{U1,I2} = p_U{}^T q_{I2}$$
$$= (0.6 \times 0.5) + (0.2 \times 0.3)$$
$$= 0.30 + 0.06 = 0.36$$

คะแนนของรายการ

- I1 = 0.44 (เคยโต้ตอบแล้ว)
- I2 = 0.36
- I3 = 0.22

ผลลัพธ์การแนะนำ

เมื่อพิจารณาเฉพาะรายการที่ยังไม่เคยโต้ตอบ
I2 มีคะแนนสูงกว่า I3
ดังนั้น ระบบแนะนำ I2 ก่อน

5 สรุปสั้น ๆ

- BPR เน้นการเรียนรู้อันดับ (ranking) มากกว่าคะแนนตรง ๆ
- เหมาะกับ implicit feedback
- ใช้คู่เปรียบเทียบ positive vs unobserved
- หลังเรียนรู้แล้ว สามารถจัดอันดับ item เพื่อแนะนำ top-N ได้

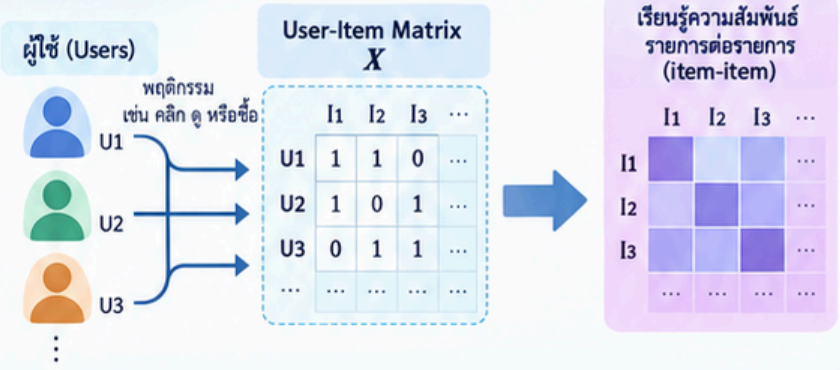
แนวคิดสำคัญ: “ของที่ผู้ใช้เคยชอบ ควรอยู่อันดับสูงกว่าของที่ยังไม่เคยโต้ตอบ”

EASE Recommendation Model

แนวคิด + ตัวอย่างการคำนวณแบบง่าย

1 EASE คืออะไร?

- เป็นโมเดลแนะนำแบบตื้น (shallow model) สำหรับ Collaborative Filtering
- ใช้ข้อมูลพฤติกรรมผู้ใช้–รายการ เช่น คลิก ดู หรือซื้อ
- เรียนรู้ความสัมพันธ์ระหว่างรายการต่อรายการ (item-item)
- **จุดเด่น:** สูตรตรงไปตรงมา ฝึกได้เร็ว และให้ผลลัพธ์ดี



2 แนวคิดหลัก

User-Item Matrix X	คำนวณ $G = X^T X$	เพิ่ม regularization: $G + \lambda I$	หา inverss: $P = (G + \lambda I)^{-1}$	คำนวณค่าน้ำหนัก $B = -P / \text{diag}(P)$, และกำหนด diag(B)=0	ทำนายคะแนน: $s_u = x_u B$
X $(m \times n)$	G $(n \times n)$	$G + \lambda I$ $(n \times n)$	P $(n \times n)$	B $(n \times n)$	s_u $(1 \times n)$

ความหมาย:

- X = เมทริกซ์ผู้ใช้-รายการ
- λ = ค่าควบคุมการ overfitting
- B = เมทริกซ์น้ำหนัก item-item
- s_u = คะแนนที่คาดการณ์สำหรับผู้ใช้ u

3 สูตรสำคัญ

1 คำนวณเมทริกซ์ความสัมพันธ์

$$G = X^T X$$

2 หาเมทริกซ์ผกผัน

$$P = (G + \lambda I)^{-1}$$

3 คำนวณเมทริกซ์น้ำหนัก

$$B = -\frac{P}{\text{diag}(P)},$$

และตั้งให้ค่าแนวทแยงของ $B = 0$

4 ทำนายคะแนน

$$s_u = x_u B$$

EASE จะเรียนรู้ว่า ถ้าผู้ใช้ชอบรายการหนึ่ง ควรให้คะแนนรายการอื่นเพิ่มขึ้นเท่าใด โดยอาศัยน้ำหนักในเมทริกซ์ B

4 ตัวอย่างการคำนวณอย่างง่าย

ข้อมูลผู้ใช้-รายการ (X)

	I1	I2	I3
U1	1	1	0
U2	1	0	1
U3	0	1	1

1 = เคยโต้ตอบ, 0 = ยังไม่เคยโต้ตอบ

1 คำนวณ $G = X^T X$

$$G = X^T X = \begin{bmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{bmatrix}$$

2 เพิ่ม regularization ($\lambda = 1$)

$$G + \lambda I = \begin{bmatrix} 3 & 1 & 1 \\ 1 & 3 & 1 \\ 1 & 1 & 3 \end{bmatrix}$$

3 หา inverse: $P = (G + \lambda I)^{-1}$

$$P = \begin{bmatrix} 0.40 & -0.10 & -0.10 \\ -0.10 & 0.40 & -0.10 \\ -0.10 & -0.10 & 0.40 \end{bmatrix}$$

4 คำนวณเมทริกซ์น้ำหนัก $B = -P / \text{diag}(P)$, และตั้ง diag(B)=0

$$B = \begin{bmatrix} 0.00 & 0.25 & 0.25 \\ 0.25 & 0.00 & 0.25 \\ 0.25 & 0.25 & 0.00 \end{bmatrix}$$

5 ทำนายคะแนนสำหรับผู้ใช้ U1

ประวัติของผู้ใช้ U1:

$$x_{U1} = [1, 1, 0]$$

$$s_{U1} = x_{U1} B = [1, 1, 0] \times B$$

$$= [0.25, 0.25, 0.50]$$

ผลลัพธ์การแนะนำ

ผู้ใช้ U1 เคยมี I1 และ I2 แล้ว
คะแนนสูงสุดของรายการที่ยังไม่เคยมีคือ
$I3 = 0.50$
ดังนั้น ระบบแนะนำ $I3$

5 สรุปสั้นๆ

- EASE ใช้การคำนวณเชิงเส้นแบบปิดรูป (closed-form solution)
- เรียนรู้จากความสัมพันธ์ item-item
- เหมาะกับ implicit feedback เช่น click, view, purchase
- หลังคำนวณ B แล้ว การทำนายทำได้ง่ายด้วย $s_u = x_u B$

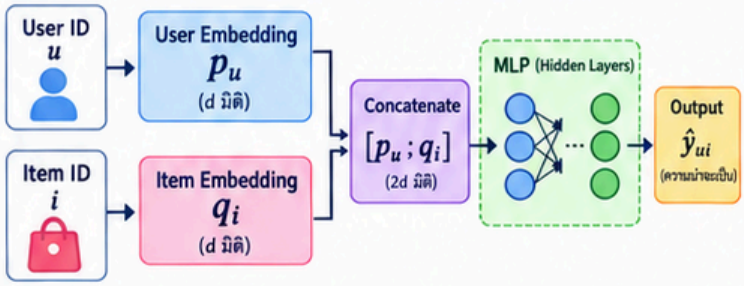
NCF Recommendation Model

แนวคิด + ตัวอย่างการคำนวณแบบง่าย

1 NCF คืออะไร?

- NCF = Neural Collaborative Filtering
- เป็นโมเดลแนะนำที่ใช้โครงข่ายประสาทเทียม เพื่อเรียนรู้ความสัมพันธ์ระหว่างผู้ใช้และรายการ
- รับ user ID และ item ID แล้วแปลงเป็น embedding
- ส่ง embedding ผ่านชั้นซ่อน (hidden layers / MLP) เพื่อเรียนรู้ปฏิสัมพันธ์แบบไม่เชิงเส้น
- มักใช้สำหรับ implicit feedback เช่น click, view, purchase
- สุดท้ายให้คะแนนความน่าจะเป็นว่าผู้ใช้จะสนใจรายการนั้น

ภาพรวมสถาปัตยกรรม NCF



ตัวอย่างข้อมูลการมีปฏิสัมพันธ์ (user-item interaction)

User	I1	I2	I3
U1	1	0	1
U2	0	1	0
U3	1	0	0

1 = มีการคลิก/สนใจ 0 = ไม่มี

2 แนวคิดหลัก

1 เก็บข้อมูล implicit feedback

User	I1	I2	I3
U1	1	0	1
U2	0	1	0
U3	1	0	0

2 แปลง user/item เป็น embedding

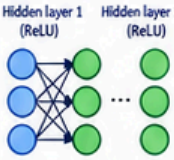
ผู้ใช้ u → p_u (d มิติ)

รายการ i → q_i (d มิติ)

3 รวมเวกเตอร์

$z_0 = [p_u ; q_i]$ (2d มิติ)

4 ส่งผ่าน MLP



5 คำนวณคะแนน/ความน่าจะเป็น

$\hat{y}_{ui} = \sigma(\text{MLP}([p_u ; q_i]))$

ค่าอยู่ระหว่าง 0 – 1

6 ฝึกด้วย loss แล้วจัดอันดับ

Binary Cross-Entropy (BCE Loss)

↓

หลังฝึกแล้ว จัดอันดับ Top-N

ความหมาย

- u : รหัสผู้ใช้ (user ID)
- i : รหัสรายการ (item ID)
- p_u : embedding ของผู้ใช้ u
- q_i : embedding ของรายการ i
- MLP : โครงข่ายประสาทเทียมหลายชั้น (hidden layers)
- $\hat{y}_{ui}$: คะแนนความน่าจะเป็นว่าผู้ใช้ u จะสนใจรายการ i

3 สูตรสำคัญ

1 Embedding

$p_u = E_u[u] \quad q_i = E_i[i]$

ดึง embedding ของผู้ใช้และรายการจากตาราง embedding

2 Concatenate

$z_0 = [p_u ; q_i]$

เวกเตอร์ขนาด 2d มิติ

3 Hidden layer (MLP)

$z_1 = \text{ReLU}(W_1 z_0 + b_1)$

$z_2 = \text{ReLU}(W_2 z_1 + b_2)$

... (อาจมีหลายชั้น)

4 Prediction

$\hat{y}_{ui} = \sigma(h^T z_L + b)$

$\sigma(x) = \frac{1}{1+e^{-x}}$

5 Loss (Binary Cross-Entropy)

$L = -\left[y \log(\hat{y}_{ui}) + (1-y)\log(1-\hat{y}_{ui})\right]$

$y = 1$ ถ้ามีการสนใจ (positive sample)

$y = 0$ ถ้าไม่มี (negative sample)

NCF ให้โมเดลเรียนรู้ปฏิสัมพันธ์แบบไม่เชิงเส้น จึงยืดหยุ่นกว่าวิธีที่ใช้ dot product เพียงอย่างเดียว

4 ตัวอย่างการคำนวณอย่างง่าย

A ข้อมูลตัวอย่าง

- ผู้ใช้ U1 เคยสนใจ I1 ($y = 1$)
- ต้องการคำนวณคะแนนของคู่ (U1, I2)
- ใช้ embedding ขนาด 2 มิติ

ค่าที่กำหนดให้

$p_{U1} = [0.6, 0.2]$

$q_{I2} = [0.4, 0.8]$

$W_1 = \begin{bmatrix} 0.5 & 0.2 & 0.4 & 0.1 \\ 0.3 & 0.6 & 0.1 & 0.5 \end{bmatrix}$

$b_1 = [0, 0]$

$h = [0.7, 0.5], \quad b = 0$

1 รวมเวกเตอร์

$z_0 = [p_{U1} ; q_{I2}]$

$= [0.6, 0.2, 0.4, 0.8]$

2 คำนวณ hidden layer

นิวรอนที่ 1 :

$0.5(0.6) + 0.2(0.2) + 0.4(0.4) + 0.1(0.8)$

$= 0.30 + 0.04 + 0.16 + 0.08 = 0.58$

นิวรอนที่ 2 :

$0.3(0.6) + 0.6(0.2) + 0.1(0.4) + 0.5(0.8)$

$= 0.18 + 0.12 + 0.04 + 0.40 = 0.74$

$z_1 = \text{ReLU}([0.58, 0.74])$

$= [0.58, 0.74]$

3 คำนวณ logit

$a = h^T z_1$

$= 0.7(0.58) + 0.5(0.74)$

$= 0.406 + 0.370$

$= 0.776$

4 คำนวณ probability

$\hat{y} = \sigma(0.776)$

$= \frac{1}{1+e^{-0.776}}$

≈ 0.685

5 แปลผล

โมเดลคาดว่า U1 มีโอกาสสนใจ I2 ประมาณ 68.5%

$\hat{y} = 0.685$ (68.5%)

6 การจัดอันดับรายการให้กับ U1

อันดับ	รายการ	คะแนน ($\hat{y}$)
1	I2	0.685 ← แนะนำ
2	I3	0.420

แนะนำ I2 เป็นอันดับแรก เนื่องจากมีคะแนนสูงกว่า

ค่า loss สำหรับคู่ฝึกนี้ (ถ้า $y = 1$)

$L = -\log(0.685) \approx 0.378$

ยิ่ง loss ต่ำ แสดงว่าโมเดลทำนายได้ใกล้เคียงกับค่าจริงมากขึ้น

5 สรุปสั้น ๆ

1 ใช้ embedding ของผู้ใช้และรายการ

แปลง user ID และ item ID เป็นเวกเตอร์ที่เรียนรู้ได้

2 ใช้ neural network เรียนรู้ความสัมพันธ์ไม่เชิงเส้น

MLP ช่วยให้โมเดลจับรูปแบบปฏิสัมพันธ์ที่ซับซ้อนได้

3 ให้คะแนนความน่าจะเป็น เพื่อจัดอันดับรายการ

คำนวณ $\hat{y}_{ui} \in [0, 1]$ แล้วจัดอันดับรายการที่มีคะแนนสูง

4 หลังฝึกแล้วสามารถแนะนำ top-N ได้

นำรายการที่มีคะแนนสูงสุดมาแนะนำให้ผู้ใช้ เช่น Top-5, Top-10

แนวคิดสำคัญ: embedding → MLP → probability → ranking